



Implementation of an efficient and scalable 5-point Stencil algorithms

15th July 2026

Devid Rosa

Data Science &
Artificial Intelligence

**High Performance
Computing**
2024–2025

Three Questions Guided the Optimization

- 01** **Where is time spent?**
- 02** **What limits one node?**
- 03** **Does it scale out?**

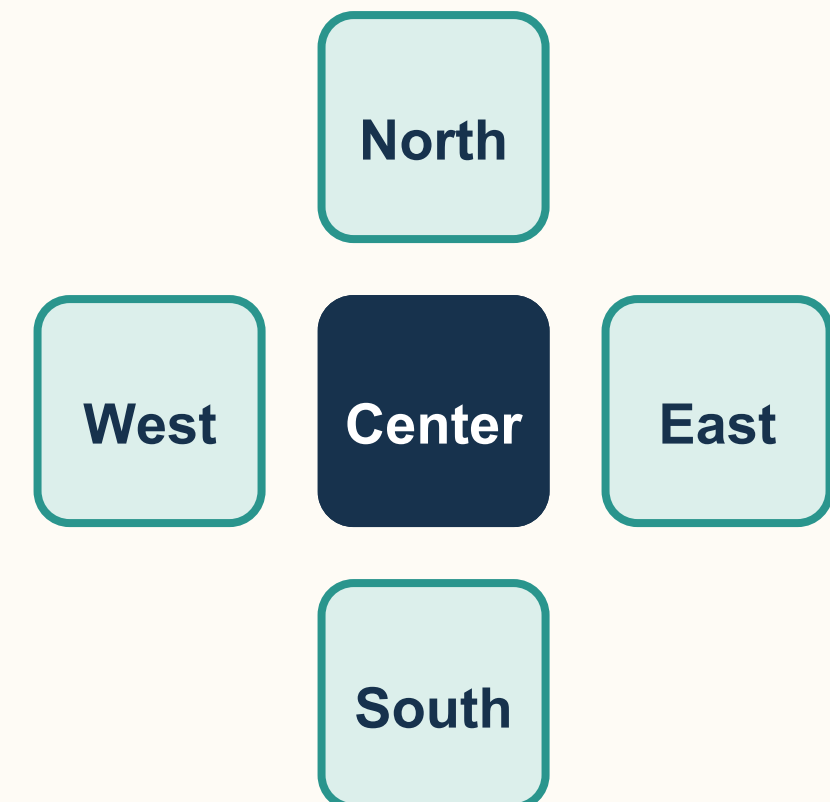
5-Point Stencil Algorithm

Heat equation discretized on a $m \times l$ grid

$$U_{m,l}^{n+1} = U_{m,l}^n + \frac{\alpha \Delta t}{\Delta x^2} (U_{m-1,l}^n + U_{m+1,l}^n - 2U_{m,l}^n) + \frac{\alpha \Delta t}{\Delta y^2} (U_{m,l-1}^n + U_{m,l+1}^n - 2U_{m,l}^n)$$

5 point algorithm:

the new value on the center depends from its old value and the value of its neighbours

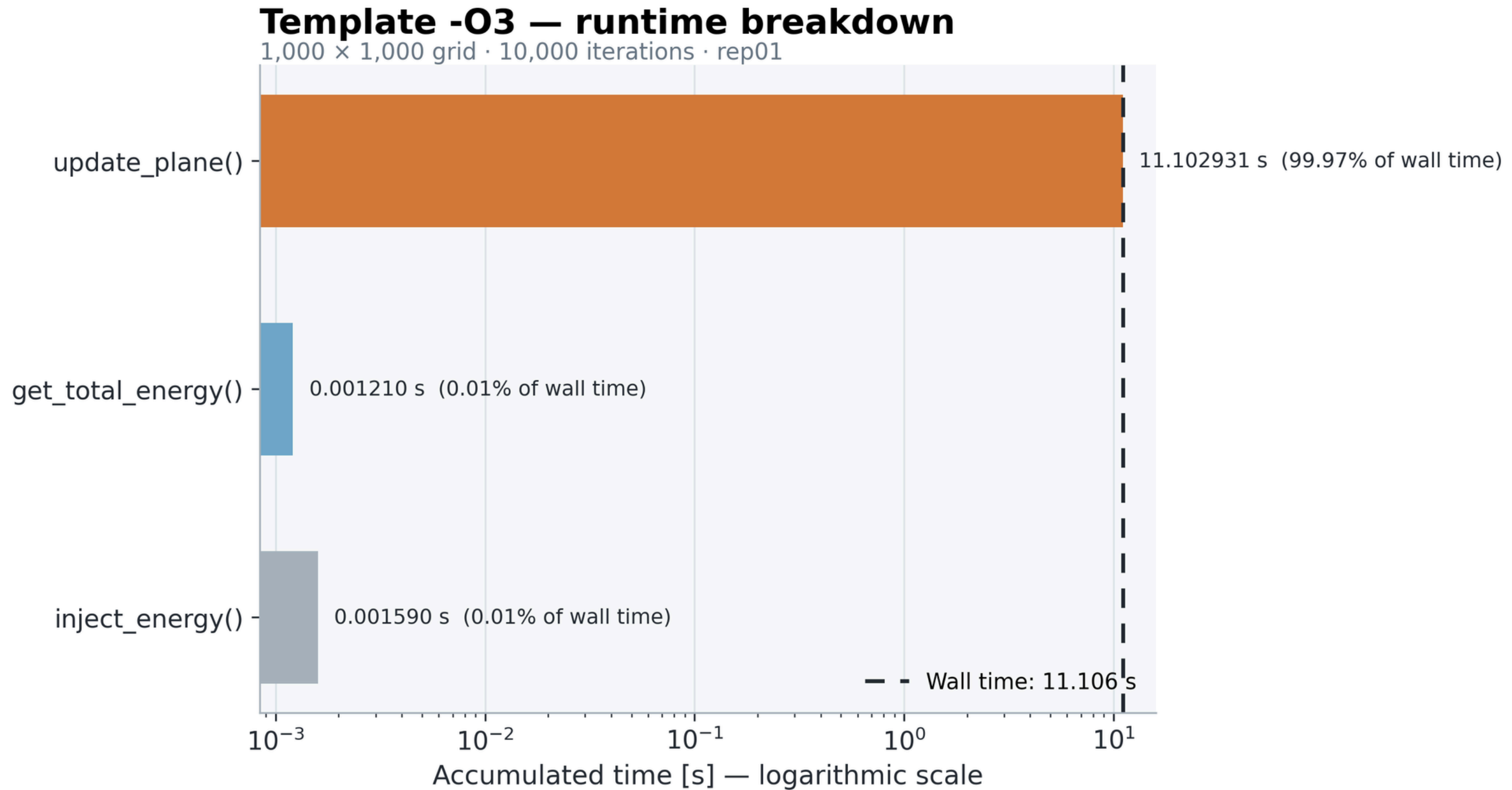


Machine description

CINECA Leonardo — Booster module

- 1× Intel Xeon Platinum 8358, 32 physical cores at 2.6 GHz
- Single socket, split into 2 NUMA domains through SNC
- 512 GB RAM
- HDR InfiniBand interconnect, Dragonfly+ topology
- 4× NVIDIA A100 GPUs (not used)
- GCC 12.2.0 and Open MPI 4.1.6

First code analysis



Initial code optimization

- Algebraic simplification
- Loop-invariant code motion
- Row-wise contiguous memory access
- restrict pointers

The 5-point stencil has very low arithmetic intensity, making it memory-bound. This memory bottleneck will be the central point of to the project.

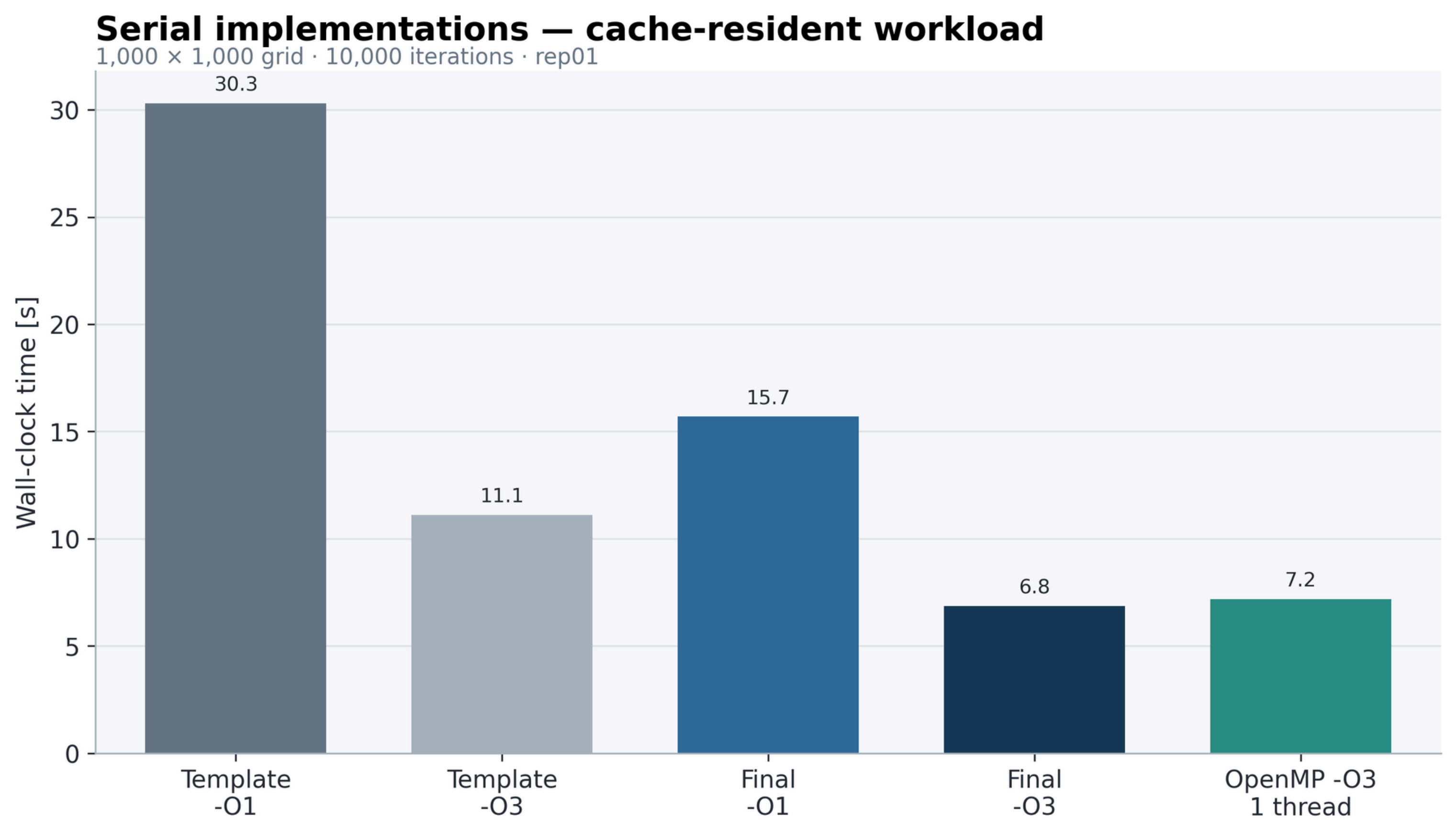
Preliminary optimizations

```
const size_t fysize = (size_t)size[_x_]+2;
const int    xsize = size[_x_];
const int    ysize = size[_y_];
const double cc = C_CENTER;
const double cn = C_NEIGH;

for (int j = 1; j <= ysize; j++) {
    const double *restrict center = old + (size_t)j *fysize;
    const double *restrict north  = old + (size_t)(j-1) *fysize;
    const double *restrict south  = old + (size_t)(j+1) *fysize;
    double *restrict result       = new + (size_t)j *fysize;

    for (int i = 1; i <= xsize; i++)
        result[i] = cc*center[i] + cn*(north[i]+south[i]+center[i-1]+center[i+1]);
}
```

SERIAL RESULTS



OpenMP primitives

```
static inline int update_plane ( )

#ifdef _OPENMP
#pragma omp parallel for schedule(static)
#endif

    for (int j = 1; j <= ysize; j++) {
        //Computation
    }

// ...

static inline int get_total_energy()

#ifdef _OPENMP
#pragma omp parallel for schedule(static) reduction(+:tot)
#endif
    for ( int j = 1; j <= ysize; j++ ) {
        const double *line = plane + (size_t)j*fxsize;
        for ( int i = 1; i <= xsize; i++ )
            tot += line[i];
    }
```

parallel for distributes independent grid rows among the threads. Since each row has the same workload, **schedule(static)** minimizes scheduling overhead

Each thread has a private partial sum and combines the results at the end

NUMA Awareness

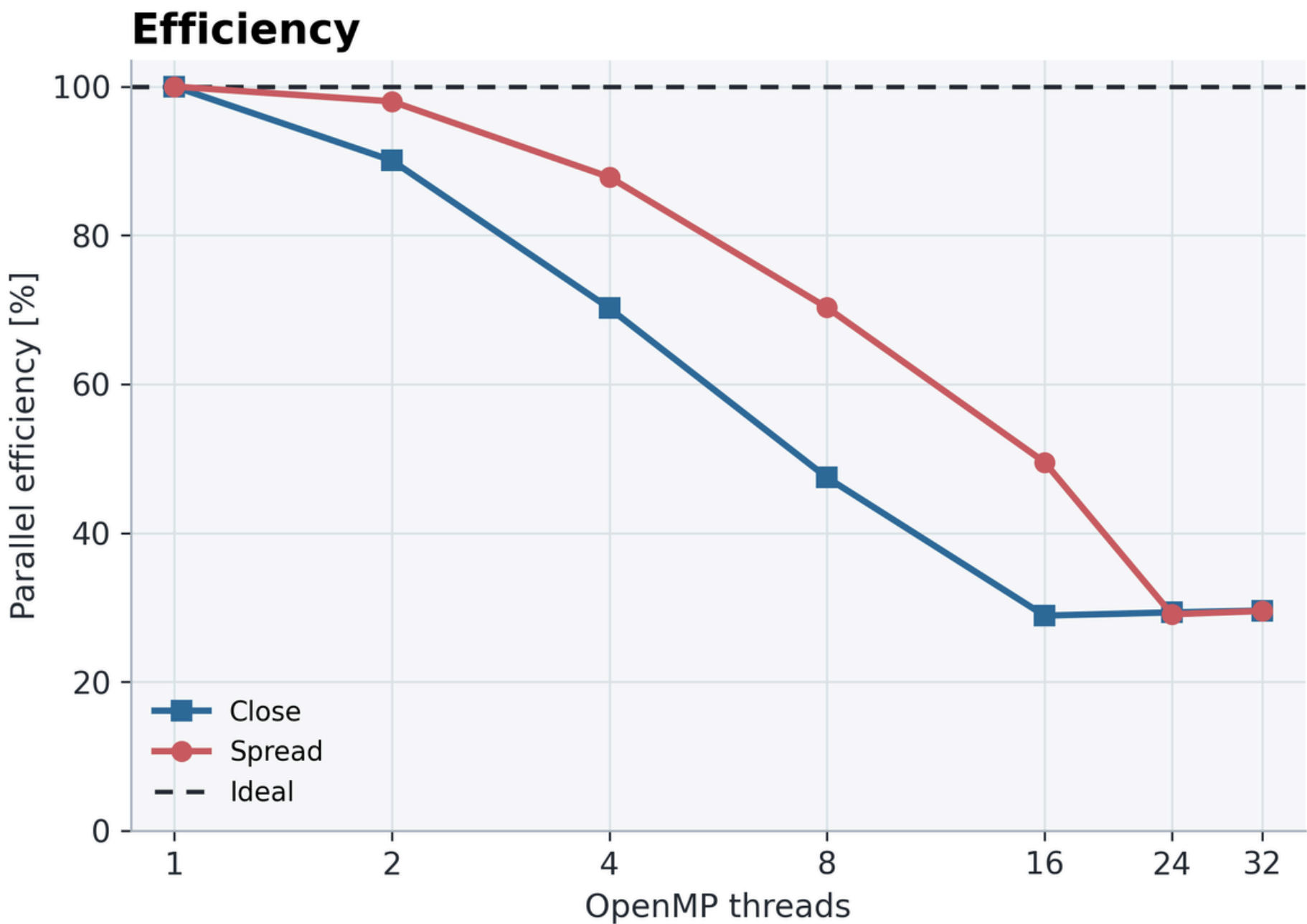
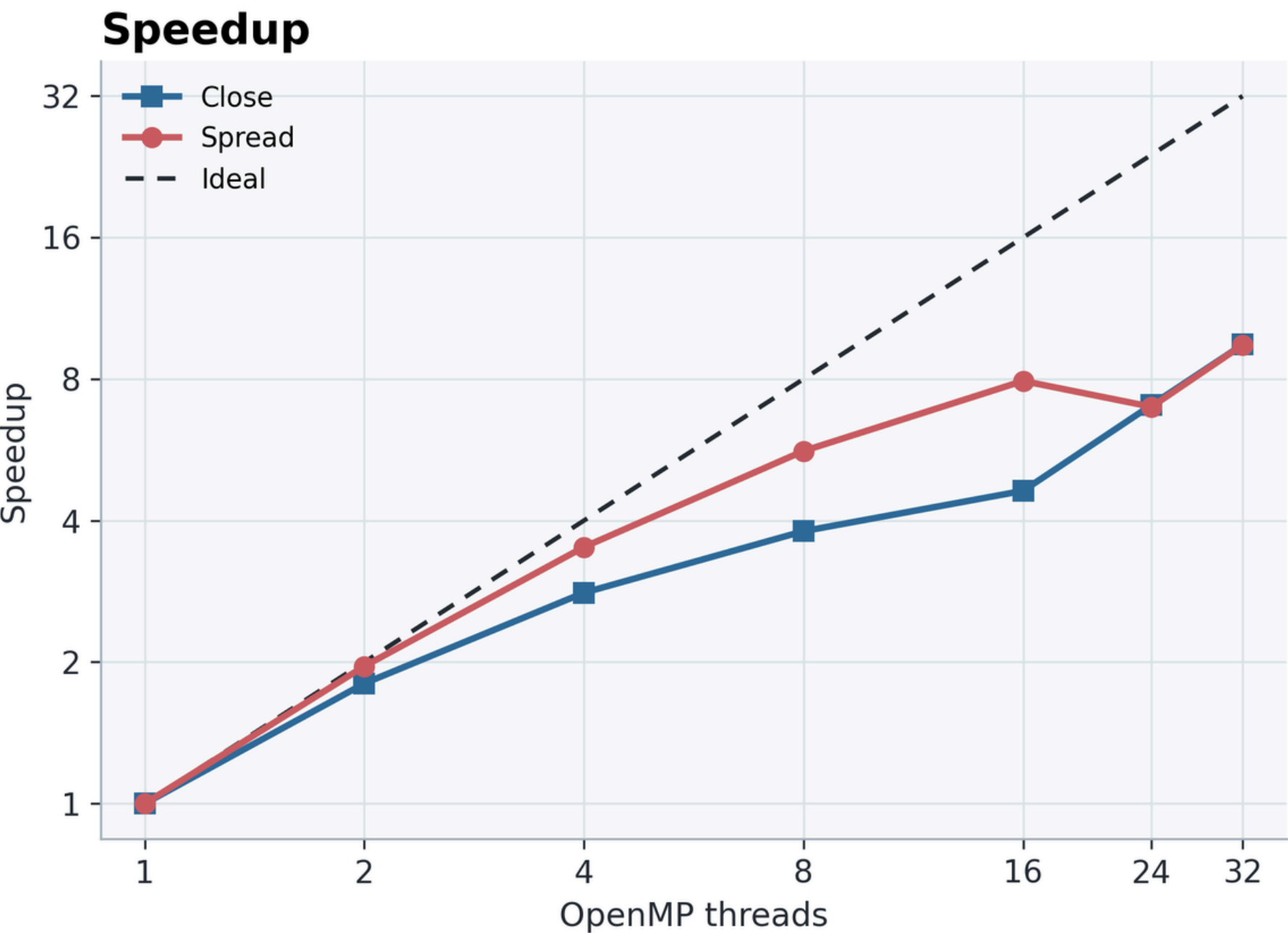
```
# Values varied across the experiments:
# nodes = 1, 2, 4, 8, 16
# ranks × threads per node = 1×32 ... 32×1

#SBATCH --partition=boost_usr_prod
#SBATCH --nodes=16
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=8
#SBATCH --hint=nomultithread
#SBATCH --exclusive

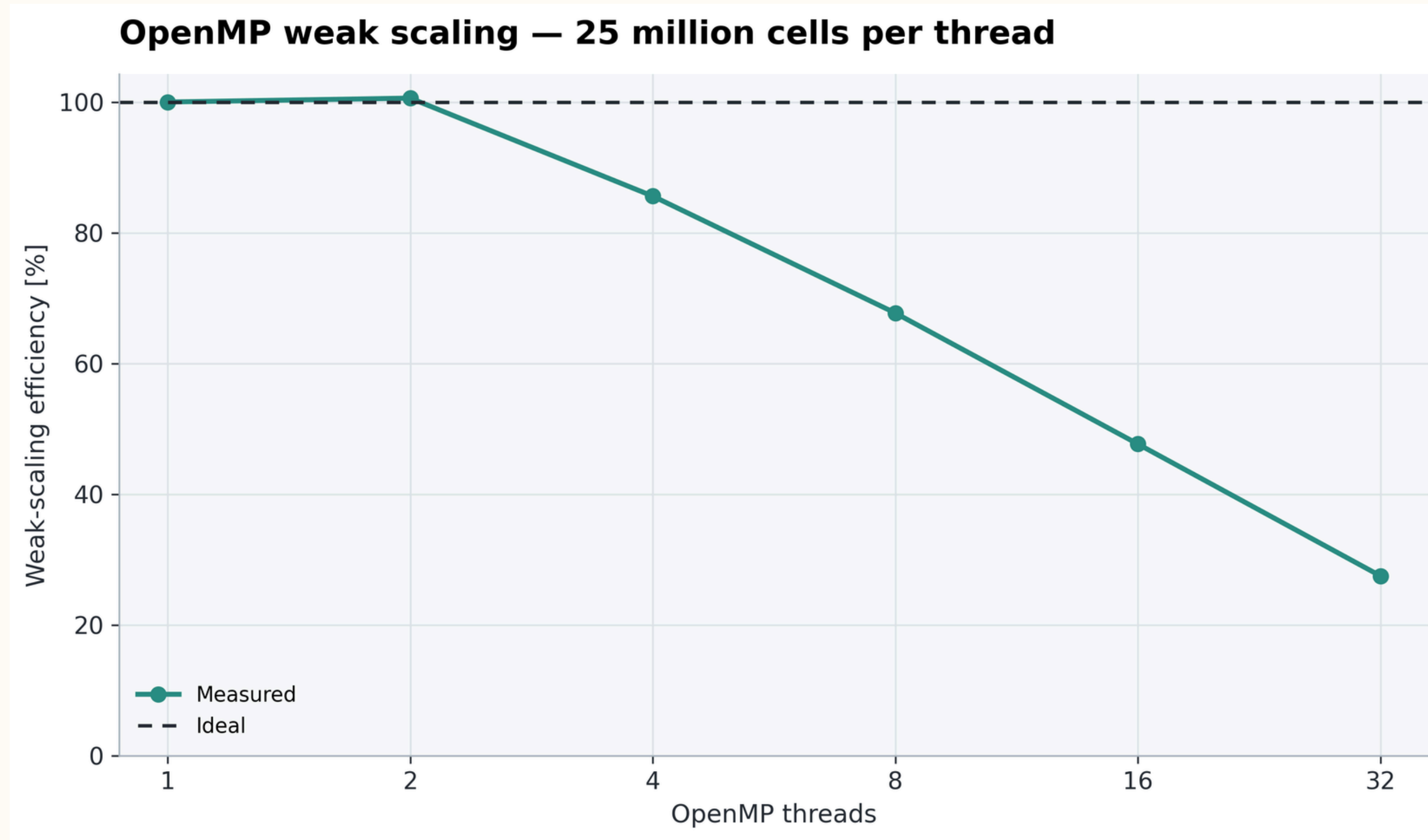
export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}
export OMP_DYNAMIC=false
export OMP_PLACES=cores
export OMP_PROC_BIND=spread
```

```
#ifdef _OPENMP
#pragma omp parallel for schedule(static)
#endif
for (size_t index = 0; index < frame; index++) {
    old_plane[index] = 0.0;
    new_plane[index] = 0.0;
}
```

OpenMP strong scaling — thread placement



OPENMP RESULTS



MPI Main Optimizations

- Near-square domain decomposition
- Halo exchange
- Minimal global communication

Near-square domain decomposition

```
for (int nx = 1; nx <= ntasks; nx++) {  
    if (ntasks % nx != 0)  
        continue;  
  
    int ny = ntasks / nx;  
  
    double local_x = (double)global_size[_x_] / nx;  
    double local_y = (double)global_size[_y_] / ny;  
  
    double score = local_x > local_y ? local_x / local_y : local_y / local_x;  
  
    if (!found || score < best_score) {  
        process_grid[_x_] = nx;  
        process_grid[_y_] = ny;  
        best_score = score;  
        found = 1;  
    }  
}
```

The process grid is selected to make local subdomains as square as possible.

This minimizes the perimeter/area ratio, reducing halo communication relative to computation

Halo exchange

```
#pragma omp parallel for schedule(static)
for (uint j = 1; j <= ysize; j++) {
    send_west[j - 1] = data[IDX(1, j)];
    send_east[j - 1] = data[IDX(xsize, j)];
}
```

East/west columns are non-contiguous in row-major memory and are packed into contiguous buffers.

```
MPI_Irecv(north_halo, ..., north, ...);
MPI_Irecv(south_halo, ..., south, ...);
MPI_Irecv(recv_west, ..., west, ...);
MPI_Irecv(recv_east, ..., east, ...);
```

North/south rows are already contiguous and can be transferred directly.

```
MPI_Isend(north_row, ..., north, ...);
MPI_Isend(south_row, ..., south, ...);
MPI_Isend(send_west, ..., west, ...);
MPI_Isend(send_east, ..., east, ...);
```

Non-blocking receives are posted first, ensuring a order-independent halo exchange.

Halo exchange

```
MPI_Waitall(nrequests,  
            requests,  
            MPI_STATUSES_IGNORE);
```

A single MPI_Waitall ensure correctness before the stencil update.

```
#pragma omp parallel for schedule(static)  
for (uint j = 1; j <= ysize; j++) {  
    data[IDX(0, j)] =  
        recv_west[j - 1];  
  
    data[IDX(xsize + 1, j)] =  
        recv_east[j - 1];  
}
```

East/west buffers are unpacked into the halo columns, while north/south data are received directly into the halo rows

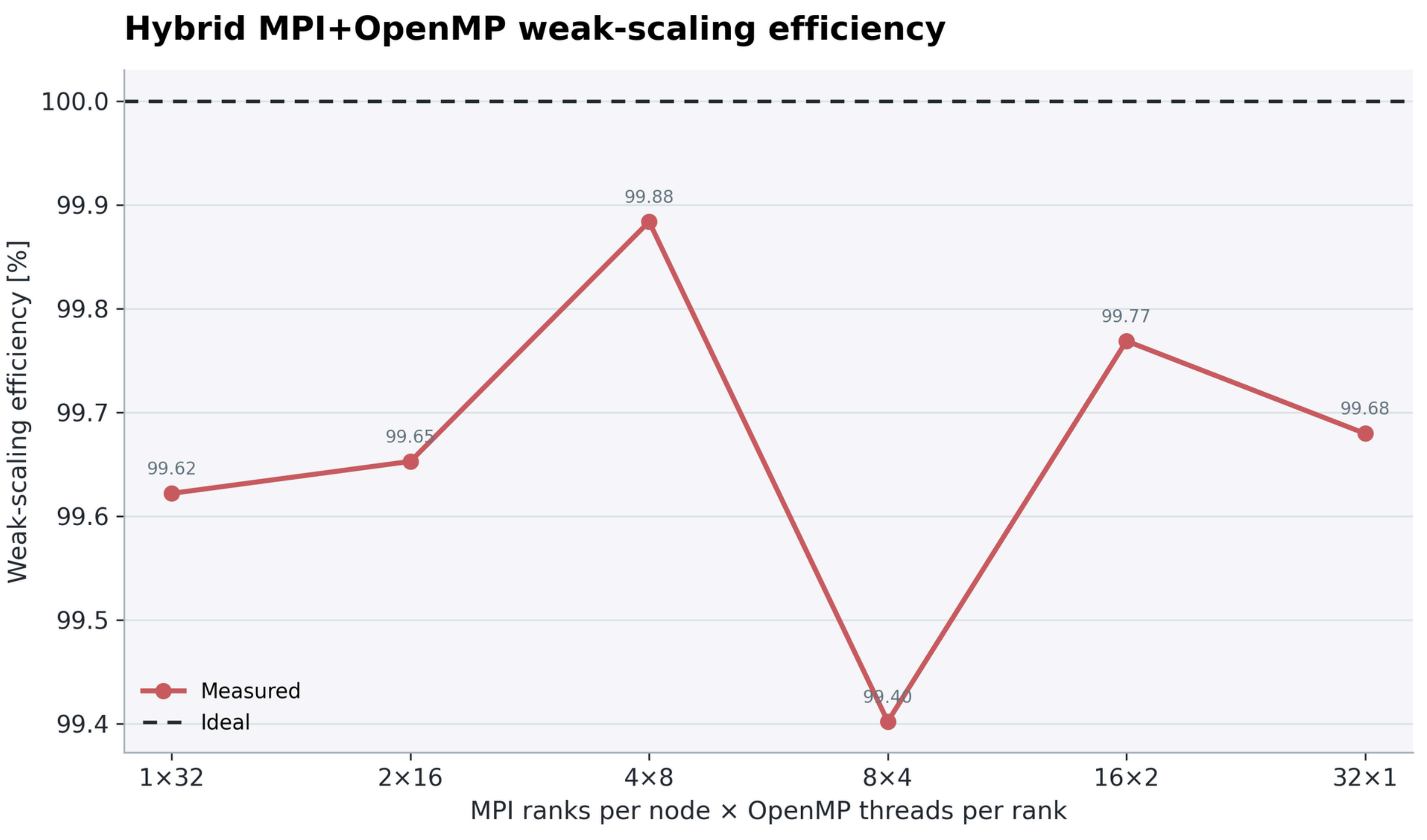
Minimal global communication

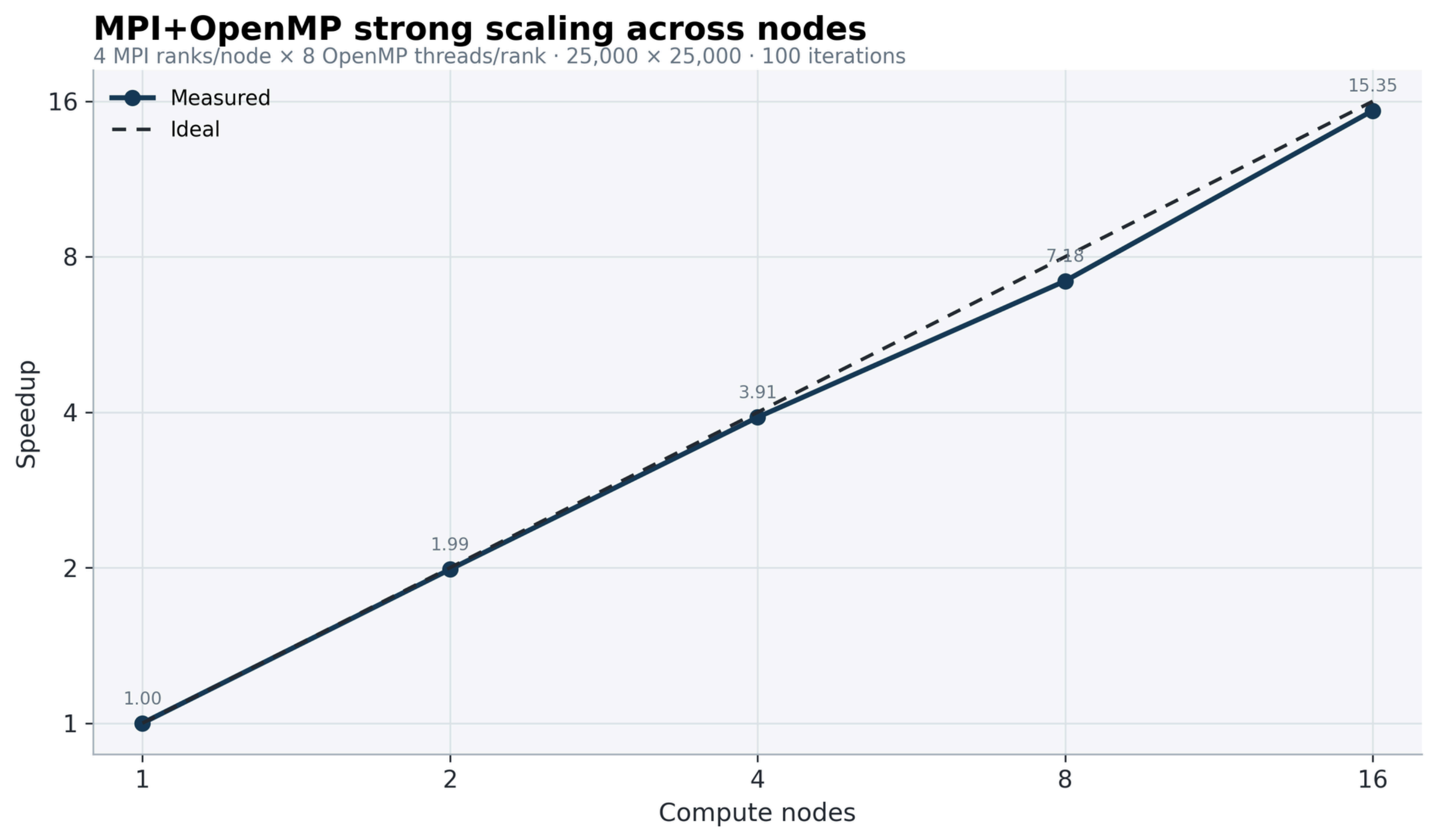
```
if (rank == 0) {  
    generate_global_sources(global_sources);  
}  
  
MPI_Bcast(global_sources,  
          nsources * 2,  
          MPI_UNSIGNED, 0, comm);
```

Rank 0 generates the global source coordinates and broadcasts them once to all ranks. Each rank then retains only its local sources.

```
double local_energy;  
double total_energy;  
  
get_total_energy(plane, &local_energy);  
  
MPI_Reduce(&local_energy,  
           &total_energy,  
           1, MPI_DOUBLE,  
           MPI_SUM, 0, comm);
```

Each rank computes its local energy contribution. MPI_Reduce sums these values, and rank 0 receives the global result.





MPI RESULTS

